UNIVERSITY OF WESTMINSTER

Informatics Institute of Technology (IIT) Sri Lanka, Affiliated with The University of Westminster, UK

5SENG003C.2 Algorithms: Theory, Design and Implementation

Module Leader: Ms. Malsha Fernando

Assignment Number: 01

Assignment Type: Individual

Deadline: 1st April 2026

Student ID: 20231489/w2120471

Student name: P.M.K.N.N.Yasasiri

A. Choice of Data Structure and Algorithm

Data Structure - Adjacency List

One of the best options I considered was to maintain a directed graph using an adjacency list, which in Java can be implemented as HashMap<Integer, Set<Integer>>. The key within this map represents a vertex, while the corresponding value stores all the vertices that are directly reachable from outgoing edges. This proved to be an excellent option because it naturally represents the directed graphs, and it is implemented using standard Java data structures. Also, it is more space-efficient than an adjacency matrix, especially for sparse graphs. Besides, it provides the basic operations that are required for this coursework, like inserting edges, retrieving neighbors, identifying sinks, and deleting vertices.

Algorithm - Sink Elimination and DFS

To determine whether a graph is acyclic or not, I implemented the sink elimination algorithm, which identifies a sink as a vertex with no outgoing edges. Essentially, it works as follows: the program keeps finding a sink, eliminating it, and repeating the process. If all the vertices are eliminated in this way, the graph is acyclic. However, if the program ends up with vertices still there but no sink is found, the graph contains a cycle. This is a good approach since every acyclic directed graph has at least one sink.

After establishing that the graph is cyclic, the program performs depth-first search (DFS) to identify and print one cycle. DFS marks the visited vertices and keeps track of the current recursion path as well. Upon encountering a vertex that is already in the recursion stack, a cycle is found. The program then traces back through the parent references to reconstruct and display that cycle. This makes the solution more comprehensive, because it not only tells whether the graph is cyclic, but also shows one actual cycle.

B. Running the Algorithm on Small Benchmark Examples

Example 1 - Acyclic Graph (benchmarks/acyclic/a_40_0.txt)

To test the program on an acyclic graph, I took a_40_0.txt file from the benchmark set. The graph was correctly loaded, and then it was processed by the sink elimination method. Throughout the program's run, the program continually discovered sinks and took them away one at a time. Because the graph had no cycle, all vertices got removed in the end. So, the output at the end was YES, which means the graph is acyclic. This case illustrates that the algorithm is correct for a graph with no directed cycles.

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.3\lib\idea
Enter the File Path (or type exit to stop):"C:\Users\pmkni\OneDrive\Desktop\Algorithms\CW\benchmarks\benchmarks\acyclic\a_40_0.txt"

Original Graph Loaded From The File: C:\Users\pmkni\OneDrive\Desktop\Algorithms\CW\benchmarks\benchmarks\acyclic\a_40_0.txt
Graph Adjacency List:
0 -> [18, 2, 4]
1 -> [19]
2 -> [22]
```

```
Running Sink Elimination...
Sink Found and Removed: 24
Sink Found and Removed: 19
Sink Found and Removed: 1
Sink Found and Removed: 8
```

```
Sink Found and Removed: 13
Sink Found and Removed: 3
All vertices removed Successfully !!

Output: YES - The graph is acyclic !!

-----------------------------------
*** You Can Test Another File Now ***
Enter the File Path (or type exit to stop):
```

Example 2 — Cyclic Graph (benchmarks/cyclic/c_40_0.txt)

For the program cyclic graph tests, I took the c_40_0.txt from the benchmark set of files. Here, the program initially removed the sinks outside the cycle. But after a few removals, the vertices were still in the graph, and no sink was found. This means the residual graph part has a cycle, and hence the final answer was NO. Subsequently, the program's DFS part was employed to find and display a valid cycle from the graph. Therefore, the program is not only capable of telling if a graph is cyclic but can also show an actual cycle when it is required.

```
Enter the File Path (or type exit to stop):"C:\Users\pmkni\OneDrive\Desktop\Algorithms\CW\benchmarks\benchmarks\cyclic\c_40_0.txt"

Original Graph Loaded From The File: C:\Users\pmkni\OneDrive\Desktop\Algorithms\CW\benchmarks\benchmarks\cyclic\c_40_0.txt
Graph Adjacency List:
0 -> [6]
1 -> [36]
2 -> [25]
```

```
Running Sink Elimination...
Sink Found and Removed: 27
Sink Found and Removed: 14
Sink Found and Removed: 19
Sink Found and Removed: 36
Sink Found and Removed: 1
Sink Found and Removed: 38
Sink Found and Removed: 11
Sink Found and Removed: 15
Sink Found and Removed: 30
Sink Found and Removed: 5
No sink found. The graph contains a Cycle !!

Output: NO - The graph is not acyclic !!
Cycle Found: 20 -> 4 -> 13 -> 32 -> 17 -> 10 -> 9 -> 33 -> 33

-----------------------------------
*** You Can Test Another File Now ***
Enter the File Path (or type exit to stop):
```

C. Performance Analysis

Theoretical Analysis - Big-O Classification

The space complexity is O(V + E) as the graph is represented through an adjacency list (V = number of vertices and E = number of edges), and only the vertices and existing edges are kept in the storage.

For the sink elimination stage:

- Initially loading the graph and recording the vertices and edges in the adjacency list will take O(V + E).
- The findSink() function needs to check each vertex to identify one without an outgoing edge; the cost might be as high as O(V) for a single search.
- On top of removing the sink vertex itself, the algorithm also eliminates all the references to that vertex from other adjacency sets. This causes the extra load per removal.

- Sink searching and vertex removal processes will be reiterated until all vertices are removed aloud. Therefore, the total runtime of this implementation is estimated to be $O(V^2)$ in the case of this simple approach on sparse graphs.

For cycle reconstruction:

- At the DFS stage, if it turns out that the graph is cyclic, then this method is used to both locate and rebuild a cycle. DFS only processes each vertex and edge once, so the complexity of this phase is $O(V + E)$.

The program's overall behavior is largely influenced by the repeated sink elimination phase; the time complexity of this implementation should be considered $O(V^2)$ approximately, whereas the memory complexity remains $O(V + E)$.

Empirical Analysis - Doubling Hypothesis

The algorithm was evaluated as the sizes of the graphs grew bigger, the first one having 40 vertices and the last one 10,240 vertices. The algorithm was running tests on acyclic as well as cyclic datasets, and they needed an average running time to compare the performance for different input sizes.

```
=== EMPIRICAL PERFORMANCE  ===
PS C:\Users\pmkni\OneDrive\Desktop\Algorithms\w2120471> Run-BenchmarkCategory "ACYCLIC Graphs" "acyclic" "a"

--> ACYCLIC Graphs:
Size (N)   Avg Time (ns)      Ratio
-----------------------------------------------
40         202992000          -
80         171134180          0.84
160        239088440          1.4
320        308795040          1.29
640        278793640          0.9
1280       362310040          1.3
2560       498450260          1.38
5120       765540920          1.54
10240      1887818000         2.47
PS C:\Users\pmkni\OneDrive\Desktop\Algorithms\w2120471> Run-BenchmarkCategory "CYCLIC Graphs" "cyclic" "c"

--> CYCLIC Graphs:
Size (N)   Avg Time (ns)      Ratio
-----------------------------------------------
40         154457000          -
80         189475420          1.23
160        235857280          1.24
320        276379620          1.17
640        281326960          1.02
1280       339775200          1.21
2560       428029680          1.26
5120       747331620          1.75
10240      1939892520         2.6
```

The data reveal some unexpected double jumps or drops in the time it takes to process a certain input size, yet the main tendency is still to go up.

I could say that the performance time goes up whenever the input size increases, which would be quite normal for a graph-processing algorithm.